1. Explicatia Detaliata a Codului (C++)

Aplicatia **InfoDex** este un dictionar interactiv de termeni de informatica (C++) dezvoltat pentru a sprijini elevii de la profilul Matematica-Informatica. Proiectul foloseste concepte fundamentale C++ si stocarea datelor in fisier (persistenta).

Structura si Concepte Fundamentale Utilizate

A. Structura de Date (struct Termen)

Se foloseste o structura numita **Termen** pentru a grupa logic informatiile legate de un singur concept:

```
struct Termen {
    char cuvant[100];
    char definitie[500];
    char exemplu_cod[1000];
    char exemplu_rezultat[200];
};
```

· Aceasta structura implementeaza conceptul de **inregistrare/structura compusa**, necesara pentru a gestiona date complexe.

· · Toti termenii sunt stocati intr-un **tablou/vector de structuri** (**Termen dictionar[100]**), iar numarul curent de termeni este retinut de variabila globala **nrTermeni**.

B. Functii (Modularitate)

Codul este impartit in functii clare, fiecare avand un singur rol, asigurand modularitatea:

```
// --- 2. FUNCTIILE DE BAZA ---

void populeazaDateStandard() {
```
Creaza un set initial de termeni, utile cand aplicatia ruleaza pentru prima data (fisierul nu exista).

```
void incarcaDictionar() {
```
Citeste datele din fisierul infodex.db. Daca fisierul nu exista, apeleaza populeazaDateStandard().

```
void salveazaDictionar() {
```
Scrie toti termenii in fisier pentru a asigura persistenta datelor.

```
void adaugaTermen() {
```
Permite utilizatorului sa introduca un termen nou.

```
void cautaTermen() {
```
Cauta un termen in vectorul de structuri si afiseaza detaliile.

```
void afiseazaTotiTermenii() {
```
Iteraza si afiseaza toate inregistrarile.

C. Controlul Fluxului (Structuri de Control)

· **Se folosesc intensiv if/else si switch pentru a gestiona meniul si deciziile logice.**

· **Se folosesc for si while pentru a itera prin dictionar si pentru bucla principala a meniului.**

D. Manipularea Sirurilor de Caractere si I/O

Functiile din biblioteca **<cstring>** (ex: **strcpy** si **strcmp**) sunt folosite pentru a copia si compara siruri de caractere (char[]).

Functiile de intrare/iesire (**I/O**) sunt tratate prin:

cin >> optiune: Pentru citirea valorilor numerice (optiune meniu).

cin.getline(): Pentru citirea sirurilor de caractere care pot contine spatii (definitii, cod, etc.).

cin.ignore(): Mecanism esential pentru a curata *bufer-ul* dupa citirea unei valori numerice, prevenind erori la apelarea ulterioara a functiei cin.getline().

.E. Persistenta Datelor (Fisiere)

Sunt folosite clasele **ifstream** (pentru citire) si **ofstream** (pentru scriere) din biblioteca **<fstream>** .

Datele sunt salvate in fisierul infodex.db. Fiecare termen ocupa 4 linii in fisier: cuvant, definitie, cod, rezultat.

2. Documentatie Proiect: README

InfoDex

***Di**ctionarul **Info**rmatic **Dex**ter*

Descrierea Scopului

InfoDex este o aplicatie in consola, scrisa in C++, conceputa ca un **dictionar interactiv** de termeni si concepte fundamentale din programarea C++.

Scopul sau principal este de a oferi elevilor de la profilul Mate-Info un instrument rapid si usor de utilizat pentru:

1.Cercetarea definitiilor clare si concise.

2.Vizualizarea imediata a unor **exemple de cod** asociate fiecarui termen.

3.Intelegerea **rezultatului asteptat** al codului respectiv.

4.Posibilitatea de a **adauga** termeni noi, extinzand astfel baza de cunostinte a comunitatii.

Aplicatia ofera o solutie digitala la nevoia de **acces rapid la informatii utile** si **sprijin reciproc** in procesul de invatare.

Publicul Tinta

Elevii de la profilul Matematica-Informatica (clasele IX-XII).

Oricine invata elementele de baza ale limbajului de programare C++ si are nevoie de o referinta rapida a terminologiei.

Modul de Utilizare

Aplicatia este bazata pe meniu si ruleaza in consola.

1.Pornire: La prima rulare, aplicatia creeaza automat un set standard de termeni (variabila, pointer, struct, for, while, functie, etc.). La rulari ulterioare, incarca datele din fisierul infodex.db.

2.Meniu Principal: Utilizatorul introduce numarul corespunzator optiunii dorite:

1. Adauga un termen nou: Introduceti cuvantul, definitia, exemplul de cod si rezultatul.

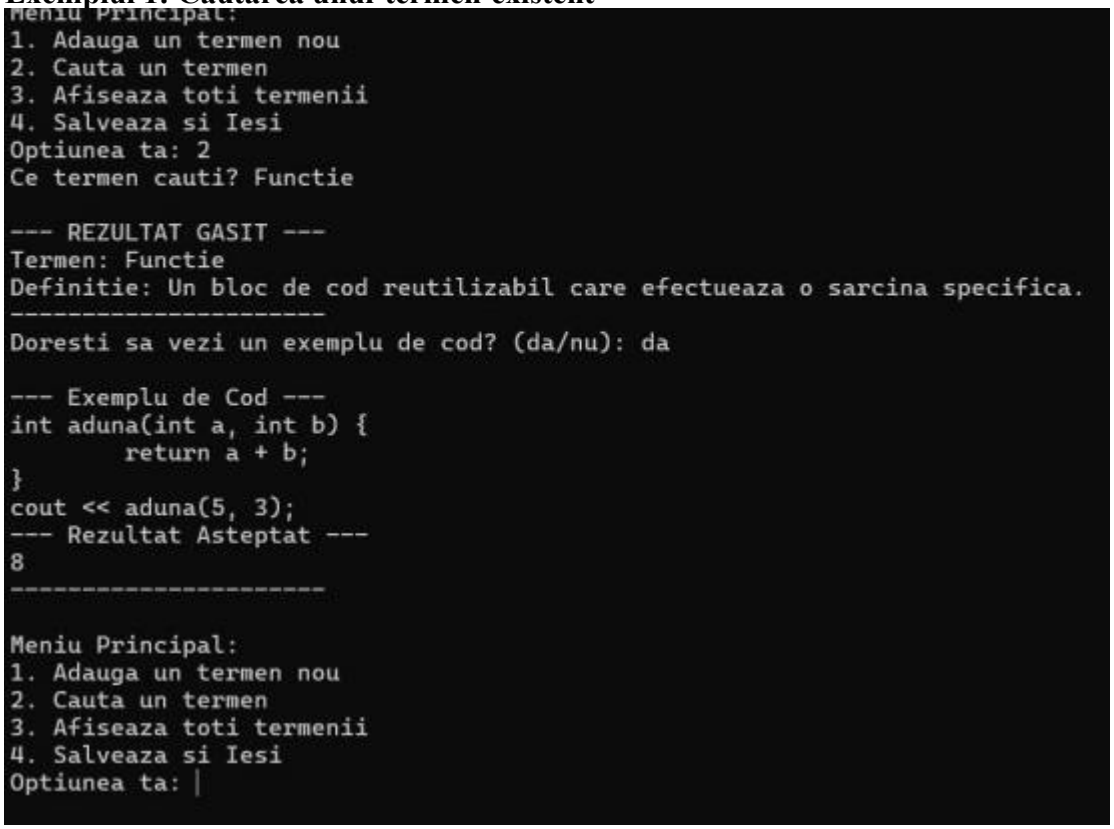
2. Cauta un termen: Introduceti termenul dorit (ex: "Pointer"). Aplicatia afiseaza definitia si ofera optiunea de a vizualiza exemplul de cod.

3. Afiseaza toti termenii: Listeaza toate inregistrarile din dictionar.

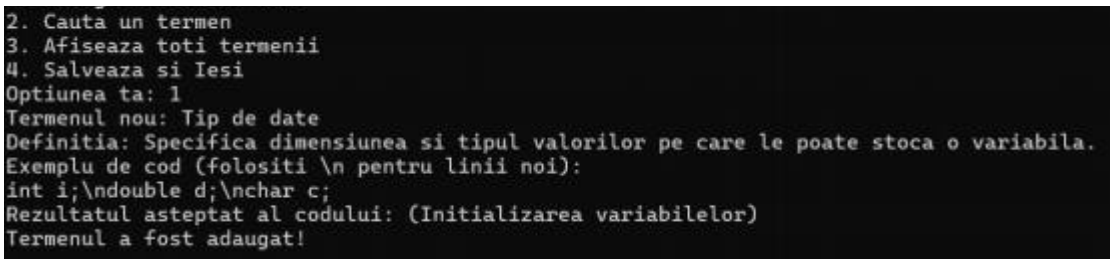
4. Salveaza si Iesi: Salveaza toate modificarile (termenii adaugati) in fisierul infodex.db si inchide aplicatia.

Exemple de Rulare

Exemplul 1: Cautarea unui termen existent



Exemplul 2: Adaugarea unui termen nou



Concluzii si Imbunatatiri Posibile

Concluzii

Aplicatia **InfoDex** indeplineste toate cerintele minime: este scrisa in C++, foloseste structuri de date, vectori, functii, structuri de control si gestionare de fisiere. Functionalitatea de adaugare si salvare permite dictionarului sa creasca organic, fiind adaptat nevoilor specifice ale utilizatorilor.

Imbunatatiri Posibile

1.Functionalitate de Stergere/Modificare: Adaugarea de optiuni in meniu pentru a sterge sau edita un termen existent.

2.Functionalitate de Quiz: Integrarea unui mod de testare interactiva bazat pe termenii din dictionar (ex: "Ghiceste termenul dupa definitie")

3.Cautare mai avansata: Implementarea unei cautari partiale (ex: afisarea tuturor termenilor care contin literele introduse).

4.Sortare: Sortarea termenilor dupa cuvant in ordine alfabetica la afisare.

5.Interfata Grafica: Trecerea de la interfata din consola la o interfata grafica (GUI) pentru o experienta de utilizare mai placuta.

COD C++:

```
/*
 * PROIECT C++: InfoDex
 * Autor: [Pirvulescu George Calin]
 * Scop: Dictionar interactiv pentru elevi (nivel 12)
 *
 */

#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

// --- 1. STRUCTURA DE DATE ---
struct Termen {
   char cuvant[100];
   char definitie[500];
   char exemplu_cod[1000];
   char exemplu_rezultat[200];
};

Termen dictionar[100];
int nrTermeni = 0;
const char NUME_FISIER[] = "infodex.db";

// --- 2. FUNCTIILE DE BAZA ---

void populeazaDateStandard() {
   cout << "Info: Se creeaza baza de date standard...\n";

   // Pentru a copia un text intr-un char[], folosim strcpy
   strcpy(dictionar[nrTermeni].cuvant, "Variabila");
   strcpy(dictionar[nrTermeni].definitie, "O locatie de memorie cu un nume, care stocheaza o valoare.");
   strcpy(dictionar[nrTermeni].exemplu_cod, "int x = 10;\ncout << x;");
   strcpy(dictionar[nrTermeni].exemplu_rezultat, "10");
   nrTermeni++;

   strcpy(dictionar[nrTermeni].cuvant, "Pointer");
   strcpy(dictionar[nrTermeni].definitie, "O variabila speciala care stocheaza o adresa de memorie.");
   strcpy(dictionar[nrTermeni].exemplu_cod, "int x = 10;\nint* p = &x;\ncout << *p;");
   strcpy(dictionar[nrTermeni].exemplu_rezultat, "10");
   nrTermeni++;

   strcpy(dictionar[nrTermeni].cuvant, "if/else");
```

```
    strcpy(dictionar[nrTermeni].definitie, "O structura de control care executa cod diferit in functie de o conditie.");
    strcpy(dictionar[nrTermeni].exemplu_cod, "int varsta = 20;\nif (varsta >= 18) {\n\tcout << \"Major\";\n} else {\n\tcout << \"Minor\";\n}");
    strcpy(dictionar[nrTermeni].exemplu_rezultat, "Major");
    nrTermeni++;


    strcpy(dictionar[nrTermeni].cuvant, "for");
    strcpy(dictionar[nrTermeni].definitie, "O bucla repetitiva folosita pentru a itera un numar cunoscut de ori.");
    strcpy(dictionar[nrTermeni].exemplu_cod, "for (int i = 0; i < 3; i++) {\n\tcout << i << \" \";\n}");
    strcpy(dictionar[nrTermeni].exemplu_rezultat, "0 1 2 ");
    nrTermeni++;


    strcpy(dictionar[nrTermeni].cuvant, "while");
    strcpy(dictionar[nrTermeni].definitie, "O bucla repetitiva care se executa atat timp cat o conditie este adevarata.");
    strcpy(dictionar[nrTermeni].exemplu_cod, "int n = 3;\nwhile (n > 0) {\n\tcout << n << \" \";\n\tn--;\n}");
    strcpy(dictionar[nrTermeni].exemplu_rezultat, "3 2 1 ");
    nrTermeni++;


    strcpy(dictionar[nrTermeni].cuvant, "Functie");
    strcpy(dictionar[nrTermeni].definitie, "Un bloc de cod reutilizabil care efectueaza o sarcina specifica.");
    strcpy(dictionar[nrTermeni].exemplu_cod, "int aduna(int a, int b) {\n\treturn a + b;\n}\ncout << aduna(5, 3);");
    strcpy(dictionar[nrTermeni].exemplu_rezultat, "8");
    nrTermeni++;


    strcpy(dictionar[nrTermeni].cuvant, "Structura (struct)");
    strcpy(dictionar[nrTermeni].definitie, "Permite gruparea mai multor variabile de tipuri diferite intr-un singur tip.");
    strcpy(dictionar[nrTermeni].exemplu_cod, "struct Elev { char nume[20]; int nota; };\nElev e1;\nstrcpy(e1.nume, \"Ana\");\ne1.nota = 10;\ncout << e1.nume << \" are nota \" << e1.nota;");
    strcpy(dictionar[nrTermeni].exemplu_rezultat, "Ana are nota 10");
    nrTermeni++;


    strcpy(dictionar[nrTermeni].cuvant, "Recursivitate");
    strcpy(dictionar[nrTermeni].definitie, "O tehnica prin care o functie se auto-apeleaza pentru a rezolva o problema.");
    strcpy(dictionar[nrTermeni].exemplu_cod, "int factorial(int n) {\n\tif (n == 0) return 1;\n\treturn n * factorial(n - 1);\n}\ncout << factorial(4);");
    strcpy(dictionar[nrTermeni].exemplu_rezultat, "24");
    nrTermeni++;


    strcpy(dictionar[nrTermeni].cuvant, "#include");
    strcpy(dictionar[nrTermeni].definitie, "O directiva preprocesor care include continutul unui fisier (header) in cod.");
    strcpy(dictionar[nrTermeni].exemplu_cod, "#include <iostream>\n// Acum putem folosi 'cout'");
    strcpy(dictionar[nrTermeni].exemplu_rezultat, "(Permite compilarea codului care foloseste 'cout')");
    nrTermeni++;


    strcpy(dictionar[nrTermeni].cuvant, "cout");
    strcpy(dictionar[nrTermeni].definitie, "Fluxul de iesire standard (console output) din C++.");
    strcpy(dictionar[nrTermeni].exemplu_cod, "cout << \"Salut\" << \" \" << \"lume!\";");
    strcpy(dictionar[nrTermeni].exemplu_rezultat, "Salut lume!");
    nrTermeni++;


    strcpy(dictionar[nrTermeni].cuvant, "cin");
    strcpy(dictionar[nrTermeni].definitie, "Fluxul de intrare standard (console input) din C++.");
    strcpy(dictionar[nrTermeni].exemplu_cod, "int x;\ncout << \"Introdu un numar: \";\ncin >> x;\ncout << \"Ai introdus: \" << x;");
    strcpy(dictionar[nrTermeni].exemplu_rezultat, "Introdu un numar: 5\nAi introdus: 5");
    nrTermeni++;


    strcpy(dictionar[nrTermeni].cuvant, "Adresa (&)");
    strcpy(dictionar[nrTermeni].definitie, "Operatorul unar '&' returneaza adresa de memorie a unei variabile.");
    strcpy(dictionar[nrTermeni].exemplu_cod, "int x = 100;\ncout << &x;");
```

```
    strcpy(dictionar[nrTermeni].exemplu_rezultat, "(Afiseaza o adresa de memorie, ex: 0x7ffc...)");
    nrTermeni++;

    strcpy(dictionar[nrTermeni].cuvant, "Dereferentiere (*)");
    strcpy(dictionar[nrTermeni].definitie, "Operatorul unar '*' (folosit pe un pointer) acceseaza valoarea de la adresa stocata.");
    strcpy(dictionar[nrTermeni].exemplu_cod, "int x = 100;\nint* p = &x;\ncout << *p;");
    strcpy(dictionar[nrTermeni].exemplu_rezultat, "100");
    nrTermeni++;

    strcpy(dictionar[nrTermeni].cuvant, "Tablou (Array)");
    strcpy(dictionar[nrTermeni].definitie, "O colectie de elemente de acelasi tip, stocate intr-o zona de memorie contigua.");
    strcpy(dictionar[nrTermeni].exemplu_cod, "int v[3] = {10, 20, 30};\ncout << v[1]; // Afiseaza al doilea element");
    strcpy(dictionar[nrTermeni].exemplu_rezultat, "20");
    nrTermeni++;

    strcpy(dictionar[nrTermeni].cuvant, "Clasa (OOP)");
    strcpy(dictionar[nrTermeni].definitie, "Un sablon (blueprint) pentru a crea obiecte in Programarea Orientata pe Obiecte.");
    strcpy(dictionar[nrTermeni].exemplu_cod, "class Masina {\n\tpublic: char marca[20] = \"Dacia\";\n};\nMasina m;\ncout << m.marca;");
    strcpy(dictionar[nrTermeni].exemplu_rezultat, "Dacia");
    nrTermeni++;
}

/*
 * Incarca dictionarul din fisier
 * FORMAT FISIER:
 * Linia 1: cuvant
 * Linia 2: definitie
 * Linia 3: exemplu_cod
 * Linia 4: exemplu_rezultat
 */
void incarcaDictionar() {
    ifstream fin(NUME_FISIER);
    if (!fin.is_open()) {
        cout << "Info: Fisierul '" << NUME_FISIER << "' nu a fost gasit.\n";
        populeazaDateStandard(); // Cream datele standard
        return;
    }
    while (fin.getline(dictionar[nrTermeni].cuvant, 100)) {
        fin.getline(dictionar[nrTermeni].definitie, 500);
        fin.getline(dictionar[nrTermeni].exemplu_cod, 1000);
        fin.getline(dictionar[nrTermeni].exemplu_rezultat, 200);

        nrTermeni++;

        if (nrTermeni >= 100) break;
    }
    fin.close();
    cout << "Info: S-au incarcat " << nrTermeni << " termeni din '" << NUME_FISIER << "'.\n";
}

/*
 * Salveaza intregul dictionar in fisier
 */
void salveazaDictionar() {
    ofstream fout(NUME_FISIER);
    if (!fout.is_open()) {
        cout << "Eroare critica: Nu s-a putut salva fisierul!\n";
        return;
    }

    for (int i = 0; i < nrTermeni; i++) {
```

```
        fout << dictionar[i].cuvant << endl;
        fout << dictionar[i].definitie << endl;
        fout << dictionar[i].exemplu_cod << endl;
        fout << dictionar[i].exemplu_rezultat << endl;
    }

    fout.close();
    cout << "Datele au fost salvate. La revedere!\n";
}


// --- 3. FUNCTIILE MENIULUI PRINCIPAL ---

void adaugaTermen() {
    if (nrTermeni >= 100) {
        cout << "Memoria dictionarului este plina!\n";
        return;
    }

    cout << "Termenul nou: ";
    cin.getline(dictionar[nrTermeni].cuvant, 100);

    cout << "Definitia: ";
    cin.getline(dictionar[nrTermeni].definitie, 500);

    cout << "Exemplu de cod (folositi \\n pentru linii noi):\n";
    cin.getline(dictionar[nrTermeni].exemplu_cod, 1000);

    cout << "Rezultatul asteptat al codului: ";
    cin.getline(dictionar[nrTermeni].exemplu_rezultat, 200);

    nrTermeni++;
    cout << "Termenul a fost adaugat!\n";
}

void cautaTermen() {
    char cuvantCautat[100];
    cout << "Ce termen cauti? ";
    cin.getline(cuvantCautat, 100);

    bool gasit = false;
    for (int i = 0; i < nrTermeni; i++) {
        if (strcmp(dictionar[i].cuvant, cuvantCautat) == 0) {
            cout << "\n--- REZULTAT GASIT ---\n";
            cout << "Termen: " << dictionar[i].cuvant << "\n";
            cout << "Definitie: " << dictionar[i].definitie << "\n";
            cout << "----------------------\n";
            gasit = true;

            cout << "Doresti sa vezi un exemplu de cod? (da/nu): ";
            char raspuns[10];
            cin.getline(raspuns, 10);

            if (strcmp(raspuns, "da") == 0 || strcmp(raspuns, "DA") == 0) {
                cout << "\n--- Exemplu de Cod ---\n";
                cout << dictionar[i].exemplu_cod << "\n";
                cout << "--- Rezultat Asteptat ---\n";
                cout << dictionar[i].exemplu_rezultat << "\n";
                cout << "----------------------\n";
            }
            break; // Oprim cautarea dupa prima potrivire
        }
    }

    if (!gasit) {
```

```
        cout << "Termenul '" << cuvantCautat << "' nu a fost gasit in InfoDex.\n";
    }
}

void afiseazaTotiTermenii() {
    if (nrTermeni == 0) {
        cout << "InfoDex-ul este gol.\n";
        return;
    }

    cout << "\n--- Toti Termenii din InfoDex ---\n";
    for (int i = 0; i < nrTermeni; i++) {
        cout << (i + 1) << ". Termen: " << dictionar[i].cuvant << "\n";
        cout << "   Def: " << dictionar[i].definitie << "\n";
        cout << "   Cod: " << dictionar[i].exemplu_cod << "\n";
        cout << "   Rez: " << dictionar[i].exemplu_rezultat << "\n\n";
    }
    cout << "---------------------------------\n";
}


// --- 4. FUNCTIA MAIN (Bucla principala) ---

int main() {
    incarcaDictionar(); // Incarcam datele la inceput

    cout << "=========================================\n";
    cout << "     Bun venit la InfoDex!\n";
    cout << "  Dictionarul tau Interactiv\n";
    cout << "=========================================\n";

    int optiune = 0;
    while (optiune != 4) {
        cout << "\nMeniu Principal:\n";
        cout << "1. Adauga un termen nou\n";
        cout << "2. Cauta un termen\n";
        cout << "3. Afiseaza toti termenii\n";
        cout << "4. Salveaza si Iesi\n";
        cout << "Optiunea ta: ";

        // Citim optiunea numerica
        while (!(cin >> optiune)) {
            cout << "Optiune invalida. Introdu un numar: ";
            cin.clear(); // Reseteaza flag-ul de eroare al 'cin'
            // Curatam buffer-ul de textul gresit (ex: "abc")
            cin.ignore(100, '\n');
        }

        /*
         * EXPLICATIE (cin.ignore):
         * Dupa ce 'cin >> optiune' citeste numarul (ex: '1'),
         * caracterul 'Enter' ('\n') ramane in buffer.
         * Functiile 'adaugaTermen' si 'cautaTermen' folosesc 'cin.getline'.
         * 'cin.getline' citeste pana la 'Enter'.
         * Daca nu curatam buffer-ul, 'cin.getline' va gasi 'Enter'-ul
         * ramas si va returna un string gol.
         *
         * 'cin.ignore(100, '\n')' curata buffer-ul,
         * eliminand 'Enter'-ul ramas.
         */
        cin.ignore(100, '\n');

        switch (optiune) {
            case 1:
                adaugaTermen();
```

```
            break;
        case 2:
            cautaTermen();
            break;
        case 3:
            afiseazaTotiTermenii();
            break;
        case 4:
            salveazaDictionar();
            break;
            default:
            cout << "Optiune invalida. Te rog incearca din nou.\n";
            break;
      }
    }

    return 0;
}
```